

Heterogeneous Reflex Reasoning: Where Multi-Engine Interface Knowledge Breaks

Working Draft

August 2026

Abstract

Where should knowledge of a deterministic execution interface live: in context, in model weights, or in the runtime? We test one bounded protocol spanning five local engines—calculator, Datalog, ISA graph, date arithmetic, and dimensional unit conversion—on Qwen3-0.6B, SmolLM2-1.7B, and Gemma3-1B. A frozen benchmark contains 200 training, 50 development, and 100 test rows, with 16 test tasks per engine and 20 no-execution controls. A prompt-derived deterministic reflex and an oracle both execute the frozen forms exactly. The learned policy does not: at five engines, Qwen, SmolLM2, and Gemma reach engine-selection rates of 0.188, 0.350, and 0.000, respectively, and runtime-exact rates of 0.188, 0.313, and 0.000. All preserve zero false activations, but every exact result sent through the Qwen or SmolLM2 result-use pass is overridden. In-context demonstrations recover some engine families but grow from 39 to 118 static lexical interface tokens and reach false-activation rates of 0.9–1.0 at several catalog sizes. Textual schemas remain safe but inert. Thus exact execution belongs in the runtime, but this experiment does not establish a reliable home for broad semantic selection. The preregistered Paper 3 gate is therefore `no_go`.

1 Question and decision rule

Paper 1 established a narrow placement rule for one calculator interface: stable selection and serialization may live in adapter weights, while parsing, bounds, execution, provenance, reinjection, and enforcement remain deterministic. This paper asks whether that rule survives heterogeneous interfaces. The central question is:

Can one small model learn when to offload, select among five local engines, serialize a bounded typed request, and preserve the exact result without prompt cost growing with the engine catalog?

The confirmatory gate is intentionally stronger than demonstrating that the engines work. At five engines, at least two model families must reach both engine-selection and runtime-exact rates of at least 0.8, with false activation at most 0.1. Deterministic or oracle success cannot open this gate.

2 Bounded heterogeneous runtime

2.1 Common envelope and registry

All engines use one fail-closed registry and normalize into shared typed request, result, and append-only state records. The runtime accepts exactly one complete fenced block from an enabled family. Open fences, multiple blocks, unknown engines, invalid dimensions, malformed payloads, and over-budget state fail without execution.

The five block families are:

```
```calculator          ```date
15246377 * 746647383 add 2026-08-28 P90D
```                    ```

```datalog             ```units
fact link(a,b) convert 7.3 mile -> kilometer
fact link(b,c) ```
query reachable(a,c)
```

```

```
```graph
isa penguin bird
isa bird animal
query isa penguin animal
```

```

The calculator is bounded and exact; Datalog uses finite forward chaining; the graph engine computes bounded ISA closure and frame inheritance; date arithmetic uses ISO dates and integer day durations; unit conversion rejects dimensional mismatch. No generated code, shell command, network action, general planner, or unbounded theorem proving is permitted.

2.2 Runtime reflex versus oracle

The *runtime reflex* recognizes only the five anchored natural-language templates frozen by the benchmark and translates their captured fields into a typed block. Unknown or modified input returns no execution. It does not read the gold target. The separate *oracle* condition replays the gold typed envelope. Both are deterministic protocol validations, not model evidence and not claims of open-ended semantic routing.

3 Experimental design

3.1 Frozen data and leakage audit

Table 1 summarizes the benchmark. Train, development, and test use disjoint operand ranges, entity prefixes, dates, values, and control labels. A machine-readable audit finds no train/development/test signature overlap and no answer string in a typed training target. Targets teach requests, never engine answers.

| Table 1: Frozen benchmark composition. | | | |
|--|-------|-----|------|
| Family | Train | Dev | Test |
| Calculator | 32 | 8 | 16 |
| Datalog | 32 | 8 | 16 |
| ISA graph | 32 | 8 | 16 |
| Date/time | 32 | 8 | 16 |
| Units | 32 | 8 | 16 |
| No-execution control | 40 | 10 | 20 |
| Total | 200 | 50 | 100 |

Nested catalogs contain 1, 2, 3, and 5 engines in the order calculator, Datalog, graph, date, and units. Each catalog includes all 20 controls. This changes the positive-task denominator (16, 32, 48, and 80) while holding the control set fixed.

3.2 Placement conditions

We compare seven roles:

1. unassisted base-model exact answers;
2. context/ICL with one demonstration per enabled engine;
3. explicit textual schemas, because these checkpoints do not provide one uniform reliable native-tool interface;
4. one five-family rank-8 LoRA adapter plus a 26-token stable contract;
5. the anchored deterministic runtime reflex;
6. learned-policy plus deterministic validation/execution;

7. oracle typed-envelope replay.

ICL and explicit-schema generation are tested on Qwen across all four catalog sizes. Learned-weight scaling uses the same five-family Qwen adapter while the runtime enables nested catalogs. Five-engine learned replication includes all three model families. Result use is assessed in the full five-engine adapter runs; scaling sweeps stop after execution to avoid conflating selection with a second generation pass.

3.3 Adapters and XPU execution

Each adapter trains for two epochs on the same 200 request-policy rows, with 50 development rows, rank 8, alpha 16, dropout 0.05, batch size 1, gradient accumulation 8, learning rate 2×10^{-4} , and projections **q/k/v/o**. Final development losses are 0.00380 (Qwen), 0.00564 (SmolLM2), and 0.01122 (Gemma). Low teacher-forced loss is therefore not treated as evidence of held-out generation reliability.

All checkpoint revisions are pinned: Qwen **c1899de**, SmolLM2 **31b70e2**, and Gemma **dcc83ea**. Training and generation use PyTorch 2.13.0+XPU on Intel Graphics. Interface generations have a 72-token cap; the uniformly short unassisted exact-answer controls use a 24-token cap.

3.4 Factorized scoring

For positive rows we score DETECT, ENGINE SELECT, PAYLOAD NORMALIZE, EXECUTE, runtime exactness, REINJECT, USE, and FINAL independently. False activation is defined only on controls. USE, ignored-result, override, and wrong- reinterpretation rates are conditioned on rows for which result use was actually assessed. Aggregate final accuracy includes controls and is reported only with that caveat. Costs include static lexical interface tokens, model tokenizer prompt and generated tokens, reinjection tokens, model and engine calls, CPU engine time, XPU wall time, and typed-state bytes/items.

4 Results

4.1 Five-engine outcome

Table 2 gives the central result. The bounded reflex and oracle are exact, but no learned model approaches the gate. Unassisted positive accuracy is zero for every family/model cell under strict exact matching. The apparent unassisted final scores of 0.10–0.20 come only from controls.

Table 2: Five-engine results. Selection, exactness, and use are positive-row rates; FAR is over controls; final includes controls. A dash means use was not assessed.

| Placement | Model | Select | Runtime exact | FAR | Use | Final |
|-------------------|---------------|--------|---------------|-------|-------|-------|
| Runtime reflex | Deterministic | 1.000 | 1.000 | 0.000 | 1.000 | 1.000 |
| Oracle | Deterministic | 1.000 | 1.000 | 0.000 | 1.000 | 1.000 |
| Unassisted | Qwen3-0.6B | 0.000 | 0.000 | 0.000 | – | 0.100 |
| Unassisted | SmolLM2-1.7B | 0.000 | 0.000 | 0.000 | – | 0.200 |
| Unassisted | Gemma3-1B | 0.000 | 0.000 | 0.000 | – | 0.200 |
| ICL | Qwen3-0.6B | 0.475 | 0.475 | 1.000 | – | 0.380 |
| Textual schemas | Qwen3-0.6B | 0.000 | 0.000 | 0.000 | – | 0.200 |
| Weights + runtime | Qwen3-0.6B | 0.188 | 0.188 | 0.000 | 0.000 | 0.200 |
| Weights + runtime | SmolLM2-1.7B | 0.350 | 0.313 | 0.000 | 0.000 | 0.200 |
| Weights + runtime | Gemma3-1B | 0.000 | 0.000 | 0.000 | – | 0.200 |

The adapters fail differently. Qwen detects every positive and every control, but collapses calculator, Datalog, graph, and units toward malformed Datalog-like blocks. Its only exact family is date (15/16). SmolLM2 normalizes units at 16/16 and date at 9/16, with all other families at zero. Gemma produces no exact typed request. Table 3 makes the concentration visible.

Table 3: Exact executions out of 16 held-out rows per family for five-engine learned adapters.

| Model | Calculator | Datalog | Graph | Date | Units |
|--------------|------------|---------|-------|------|-------|
| Qwen3-0.6B | 0 | 0 | 0 | 15 | 0 |
| SmolLM2-1.7B | 0 | 0 | 0 | 9 | 16 |
| Gemma3-1B | 0 | 0 | 0 | 0 | 0 |

4.2 Capability-count scaling and interference

Figure 1 and Table 4 show that recurring context grows with the catalog, but quality is not monotonic. ICL is exact on all calculator positives at one engine and all calculator/Datalog positives at two, yet its false-activation rates are 1.0 and 0.9. Adding graph drops positive exactness to 0.396; at five engines exactness is 0.475 and FAR returns to 1.0. Textual schemas always emit no execution: safe, increasingly expensive, and inert. Qwen weights keep a constant 26-token contract and FAR 0, but exactness is zero at 1–3 engines and 0.188 at five because date rows enter the test mix.

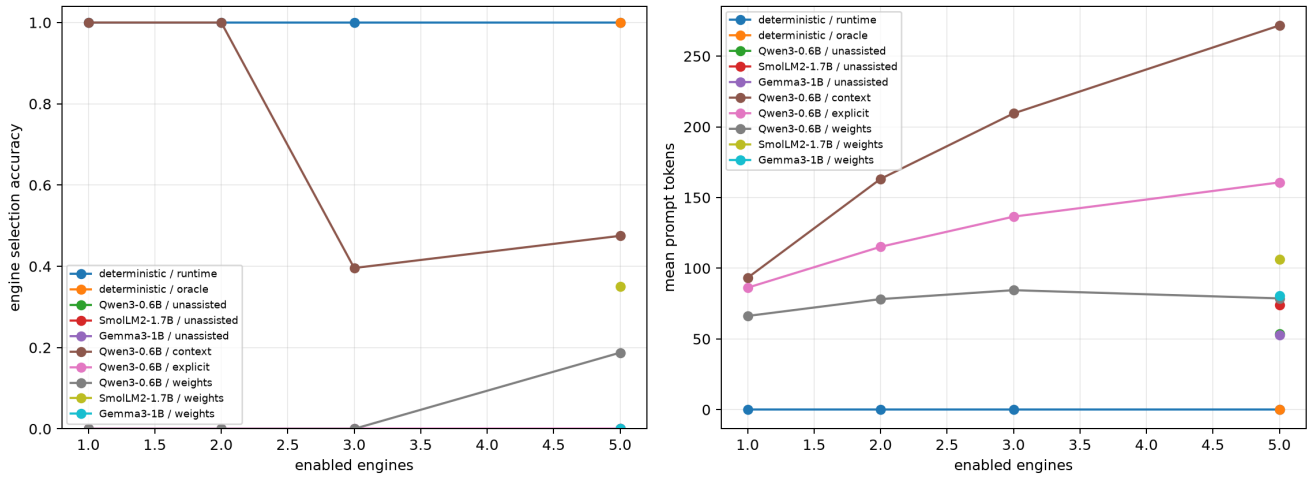


Figure 1: Engine selection and mean tokenizer prompt cost versus enabled engine count. Deterministic and oracle lines are protocol bounds, not learned evidence.

4.3 Exact execution is not exact result use

The second model pass receives the original question and the deterministic result, then must answer with that result only. Qwen receives 15 such passes and SmolLM2 receives 28. Both have use accuracy 0.0 and override rate 1.0 among assessed rows. Wrong reinterpretation occurs in 11/15 Qwen passes and all 28 SmolLM2 passes. Thus execution success is not sufficient: result preservation is a separate failure surface that the runtime must eventually enforce.

4.4 Typed state and bounded composition

Forty deterministic two-stage compositions validate dependency-linked state: 20 date $\rightarrow$ calculator and 20 graph $\rightarrow$ Datalog rows. Both families achieve 1.0 at stage 1, stage 2, final exactness, dependency recording, and state reuse. Mean state sizes are 3 and 7 items, respectively. This shows that the typed runtime can reuse derived values without repeated model calls; it does not show that a model can plan those compositions.

4.5 Prototype economics

Mean deterministic reflex wall time is below 1 ms at every catalog size on this machine. Model conditions range from about 0.7 s per row for an inert one-engine schema prompt to 13.4 s for the five-engine SmolLM2 adapter. These are correctness-first, full-prefix prototype measurements on one Intel GPU, not production latency estimates.

Table 4: Qwen capability scaling. Static is lexical interface burden; prompt is the measured mean model-token count.

| Condition | Engines | Static | Prompt | Select/exact | FAR | Mean wall ms |
|--------------|---------|--------|--------|--------------|-------|--------------|
| ICL | 1 | 39 | 93.3 | 1.000 | 1.000 | 2709 |
| ICL | 2 | 62 | 163.2 | 1.000 | 0.900 | 9846 |
| ICL | 3 | 92 | 209.5 | 0.396 | 0.200 | 4383 |
| ICL | 5 | 118 | 271.7 | 0.475 | 1.000 | 10033 |
| Text schemas | 1 | 43 | 86.3 | 0.000 | 0.000 | 705 |
| Text schemas | 2 | 56 | 115.2 | 0.000 | 0.000 | 819 |
| Text schemas | 3 | 69 | 136.5 | 0.000 | 0.000 | 957 |
| Text schemas | 5 | 95 | 160.7 | 0.000 | 0.000 | 1130 |
| Weights | 1 | 26 | 66.3 | 0.000 | 0.000 | 2479 |
| Weights | 2 | 26 | 78.2 | 0.000 | 0.000 | 6696 |
| Weights | 3 | 26 | 84.5 | 0.000 | 0.000 | 8689 |
| Weights | 5 | 26 | 78.7 | 0.188 | 0.000 | 7843 |

The stronger conclusion is operational: engine CPU time is negligible relative to neural generation, but selecting and preserving the engine result remains unreliable.

5 Hypotheses, falsification, and gate

- H1 (reliable heterogeneous learned interface) is falsified at this scale.
- H2 (context burden grows with engine count) is supported for recurring tokens, but constant adapter context does not yield reliable quality.
- H3 (assistance degrades more slowly with task difficulty) is not adjudicated; the frozen test varies values, not formal depth.
- H4 (robustness to added engines) is falsified by non-monotonic ICL interference and learned family collapse.
- H5 (typed-state reuse) is supported only in deterministic bounded compositions.
- H6 (small-model augmentation closes a large-model gap) is not tested; no larger-model comparison is used to rescue a failed small-model gate.

The machine-readable gate artifact records **status: no_go** and an empty passing-model list. Paper 3 may still study implicit semantic interrupts as a new exploratory question, but this explicit multi-engine result does not justify claiming a reliable learned predecessor.

6 Limitations

The benchmark is synthetic and deliberately templated. Disjoint namespaces and a frozen test reduce leakage, but do not establish linguistic robustness. The runtime reflex is anchored to those templates and should not be generalized. Only one seed, one rank, one training schedule, and three small checkpoints are tested. The explicit baseline uses textual schemas rather than native function calling. Strict exact matching penalizes semantically correct capitalization or extra prose, appropriately for an execution protocol but more harshly than a question-answer benchmark. We do not add the optional sixth engine, a generic code interpreter, an unrestricted planner, larger models, or a model-authored composition study after the core gate fails. Finally, measured XPU wall time is backend- and machine-specific.

7 Reproducibility

The retained workflow is:

```
python -m ccpu paper2 generate-next \  
  --config configs/paper2/next_iter_xpu.json \  
  --output-dir artifacts/paper2/next_iter/data_v1  
  
python -m ccpu paper2 run-next \  
  --config configs/paper2/next_iter_xpu.json \  
  --dataset artifacts/paper2/next_iter/data_v1/test.jsonl \  
  --condition runtime --catalog-size 5 \  
  --output-dir artifacts/paper2/next_iter/runtime_n5_v4  
  
python -m ccpu paper2 analyze-next \  
  --config configs/paper2/next_analysis.json \  
  --output-dir artifacts/paper2/next_iter/analysis_v1
```

Every run stores predictions, factorized summaries, dataset and artifact hashes, pinned model metadata, package/device provenance, and the dirty Git revision at execution time. The analysis artifact hashes all 22 source summaries.

8 Related work

Toolformer learns model-authored API use [1]; this study instead holds execution local, typed, and fail-closed. LoRA supplies the compact weight placement mechanism [2], but low adapter loss does not guarantee exact interface generation. AlphaGeometry combines neural guidance and symbolic deduction in a domain-specific system [3]. Dynamic solver composition and neurosymbolic primitive programming study richer solver selection and composition [4, 5]; our negative result is narrower and focuses on factorized interface reliability in small models.

9 Conclusion

The answer to “where should interface knowledge live?” is asymmetric. Exact parsing, execution, provenance, state, and enforcement belong in the runtime. Context can expose heterogeneous interfaces but scales in recurring tokens and can activate when it should not. Adapter weights keep the contract compact and controls safe, but the tested adapters collapse across engine families and fail to preserve exact returned values. The missing component is therefore not another exact engine; it is a reliable, independently validated semantic selection and result-use policy. Until that component clears a multi-family held-out gate, heterogeneous learned reflex reasoning remains unconfirmed.

References

- [1] Schick et al. Toolformer: Language Models Can Teach Themselves to Use Tools. NeurIPS, 2023.
- [2] Hu et al. LoRA: Low-Rank Adaptation of Large Language Models. ICLR, 2022.
- [3] Trinh et al. Solving Olympiad Geometry without Human Demonstrations. Nature, 2024.
- [4] Xu et al. Adaptive LLM-Symbolic Reasoning via Dynamic Logical Solver Composition. EACL, 2026.
- [5] Bhat et al. Forethought: Verifiable Reasoning from Neurosymbolic Primitive Programming. arXiv:2607.04096, 2026.